

thunk — North-Star Design Spec

Date: 2026-07-06 **Status:** Design (brainstorming output). Working name; renameable in one command.

Author of record: Penn Porterfield (design captured collaboratively) **Related:** `~/dev/rust-spi-tinydoom` (the kind of hardware capstone `thunk` teaches toward). Fresh project — deliberately **not** built on `~/dev/peek`.

1. NAME

`thunk` is a real term in low-level code: a small piece of deferred work, set aside now and run later, which fits the audience without editorializing. It also just sounds like a folk past tense of *think* (think, thank, thunk), which keeps a heavy subject unpretentious. Working title; renameable.

2. ONE-LINE SUMMARY

A free, self-contained systems course for justice-impacted learners: one offline Rust program that teaches the low level from true zero, up through the kernel, Rust, the SPI bus, and driving a display, ending with DOOM running on a panel the learner drove from the metal up.

3. WHAT THIS IS (AND IS NOT)

- **It is** a course-first, self-paced systems curriculum delivered as one offline artifact, with a simulator so every learner reaches the hardware finale without owning hardware.
- **It is not** a game with a lesson attached. No pet, no permadeath, no mascot.
- **It is not** a rewrite of PEEK. It borrows PEEK's proven *constraints* (offline, static, review-safe, Rust), not its code or concept.

4. WHY IT EXISTS (THE WEDGE)

Verified 2026-07-06; date-stamp before reuse: - **No prison or reentry program teaches systems/embedded/kernel.** They teach web (The Last Mile → JS/React) or Python (Justice Through Code). White space. - **The reason the space is empty is the reason this design works.** Low-level teaching normally needs hardware that will not clear facility review. A **simulator-first** artifact gets in anyway. - **The frontier is current.** Linux 7.1 stable (Jun 2026); Rust concluded its "experiment" and is now core kernel (merged 7.0, Dec 2025) — yet still **no in-tree Rust SPI abstraction** as of 7.2-rc2. - **Market direction.** Entry-level web dev is oversupplied; embedded/systems roles are short-staffed and pay more (~\$137k–\$169k). Rust is the *most-admired* language (not the highest-paid — do not overclaim).

5. AUDIENCE & DEPLOYMENT TIERS (THE SEAM)

One curriculum, **two build profiles** selected at build time: - **Inside build** — locked-down machines. Air-gapped, review-cleared, simulator-only, no external links, no network, no crypto, no exploit vocabulary. One

static binary and/or static web bundle. - **Open build** — reentry / anyone. Same course, downloadable, plus an **optional hardware capstone kit** (a small SBC + SPI display + Saleae) for learners who can get it.

Goal: the *same* experience reaches the DOOM finale whether the panel is **simulated** (inside) or **real** (open). Sim-or-real behind one interface is the keystone.

Reentry timing (the framing). The program targets the reentry window — the months just before release and just after. Inside build: learn the fundamentals air-gapped. Open build: the day you walk out, including the M6 first-contribution bridge (§8). "Justice-impacted" is the audience; *reentry* is the moment the design is built around.

6. ENTRY LEVEL & PEDAGOGY

- **True zero.** No prior programming assumed. The confident can move fast; no one is locked out.
- **Self-paced, competency-based** (WGU model): mastery over seat-time.
- **Competency gates + placement.** A short diagnostic at entry takes stock of what a learner already knows, because some arrive with real systems experience. Advancement is mastery-gated: passing a module's checks unlocks the next. The placement diagnostic lets an experienced learner test out of modules they already know instead of sitting through them. The machinery already exists in `think-core ( Check , Progress::module_mastered )`; placement reuses it. Nobody is held back, and nobody is bored.
- **Self-serve core that cannot get stuck**; an **optional facilitator kit** (pacing, answer keys, cohort/progress view) layers on. Degrades to zero staff.
- **Engagement = mastery + spaced recall + visible progress.** No mascot.

7. THE ARTIFACT — MODES

One Rust project, one content source, three ways in: - **CLI** — minimal, scriptable. Read a lesson, answer a check, see progress. - **TUI** — the primary offline classroom: paginated lessons, inline checks, the simulator's trace/panel views, progress. Stock terminal. - **Local web GUI** — the richer visual mode. Static offline bundle or `think serve` bound to `127.0.0.1` only. Where the simulated logic-analyzer trace, the panel, and DOOM show best. No external assets.

All three render from one content pipeline (lessons + checks + generators); nothing authored twice.

8. CURRICULUM LADDER (TRUE ZERO → SPI/DOOM)

Bottom-up. Each module = lessons + checks (banked + procedurally generated) + a simulator beat. - **M0 • Power On** — what a computer is, what a program is; first steps from nothing. - **M1 • The Kernel** — programs vs OS, syscalls, files & devices, mmap, drivers. - **M2 • Rust for the Metal** — ownership, `no_std`, why Rust down here. - **M3 • The Bus** — SPI: how a peripheral talks. Pairs with the simulator's trace viewer. - **M4 • The Panel** — framebuffer, mmap the buffer, driving a display. - **M5 • DOOM** — DOOM boots on the panel over the bus the learner built; then it decodes a real Rust SPI/DOOM demo (`rust-spi-tinydoom`-style). - **M6 • Intro to Open Source** (*open build*) — what open source actually is and why it matters here: licenses, version control in the open, how a project and its community work, reading

a diff, the norms of a good contribution, and the culture of merit. The on-ramp, taught before anyone sends a line of code. Its value proposition: *compiling a bright future through open source* (see `~/dev/thunk/docs/thunk-value-proposition.html`). - **M7 · First Patch** (*open build only — needs connectivity*) — the reentry bridge. Version control in the open, finding a good first issue, patch etiquette (`git send-email`, `Signed-off-by`, reviewers, the mailing list), and building a public contribution history that speaks for a person when a background check might not. Small first (a docs fix, a real bug in a tool they use), with a Linux kernel patch as the north star. It runs as a small peer community doing small, well-scoped kernel fixes together, plugging into established on-ramps (kernelnewbies.org, the Linux Kernel Mentorship Program, Outreachy) rather than reinventing them. The core pitch is **meritocracy**: the review judges the patch, not the person. Nobody on the mailing list asks about your record; a correct patch gets merged and your name stays on it. Framed honestly — the culture is exacting and blunt, but it is blind to your past and ruthless only about your code. **This module is internet-facing and therefore its own sub-spec later** — different constraints from the air-gapped core; the inside build ships without it.

9. THE SIMULATOR SUBSYSTEM (THE KEYSTONE, NET-NEW)

A pure-Rust, deterministic, embedded software model of an **SPI bus**, an **ILI9341-class panel**, and a **DOOM framebuffer**. Requirements: - No hardware, no network — runs on the inside build. - A **trace view** that echoes a logic analyzer (the Saleae experience) so learners see the bus. - The **same interface** whether backed by simulation or (open build) a real Saleae + panel over `spidev`. This trait boundary is the sim-or-real seam. - Ends by rendering DOOM to the simulated panel.

10. REVIEW-SAFETY REQUIREMENTS (FIRST-CLASS, NON-NEGOTIABLE)

1. Single self-contained artifact (static-musl binary and/or static web bundle); no installer, no admin.
2. No network at runtime; no sockets; no auto-fetched resources. `thunk serve`, if used, binds `127.0.0.1` only.
3. No cryptography, no exploit vocabulary. A `vocab-lint` CI gate enforces course-appropriate language.
4. Framing is engineering, not hacking, in all prose and UI copy.
5. Inside build compiles with **no** hardware/network code paths present — not merely disabled.
6. Reproducible, auditable builds; source public (MIT/Apache-2.0).

11. FUNDING & PARTNERSHIPS (HONEST — NOTHING SECURED)

- **Free + open-source**, aiming for sponsor/grant support.
- **Saleae** — intended outreach, not an existing program. No nonprofit/donation program exists; they offer a student discount (Logic 8 ≈ \$249 vs \$499) and a **\$200 project-writeup bounty**. The bounty is collectable now; a partnership is an ask.
- **Next Chapter** — incubator, first cohort, the justice-impacted mission tie-in the capstone requires.
- **The Last Mile / others** — potential distribution once proven; they teach web, so `thunk` is complementary.

12. AESTHETIC DIRECTION (OPEN, TO DESIGN LATER)

Clean monospace, restrained palette, the quiet confidence of good documentation and a good oscilloscope. Not neon. Detailed in a later design pass.

13. PHASING

- **Phase 1 (Spine): built and tested.** Content pipeline → CLI + TUI, M1 authored end-to-end, and a first simulator slice that boots a splash on a simulated panel. Runs today; ready for the Aug 2026 Next Chapter demo.
- **Phase 2 (Keystone): complete 2026-07-12.** Protocol-accurate simulator (real ILI9341 command set over the SPI bus model) with an annotated trace view (TUI and CLI `sim --trace`) and a deterministic corridor-scene finale rendered through the Display driver (playable DOOM deferred to the open build); the offline web GUI (static site via `thunk web`, `thunk serve` bound to 127.0.0.1 only); full M0–M6 curriculum (31 lessons, 93 checks); competency gates and a 21-item placement diagnostic. 118 workspace tests passing.
- **Phase 3 (Program): in progress.** Built 2026-07-13: the M7 First Patch module (open build), the facilitator kit (`thunk kit` + `thunk progress --export`), and the inside/open build profiles (inside = default build, hardware crate absent from the graph, gated by scripts/profile-audit.sh + CI). Remaining: playable DOOM on the open build (needs network assets), Saleae/partner outreach, first outside cohort, first facility conversation.

14. OPEN QUESTIONS

- Workspace shape: single feature-gated binary, or a crate set (`thunk-core`, `thunk-sim`, `thunk-tui`, `thunk-web`)? (Leaning: crate set — clean domain / simulator / render separation.)
- M0 on-ramp: does true-zero need a runnable code sandbox, or is read-and-check enough given the no-arbitrary-execution constraint of an inside build?
- Repo/identity: `github.com/melonmelonz/thunk`, public from day one?
- M6 (First Patch): how much can be practiced offline inside (a local git + a simulated review) vs. only for real on the open build? What partner (a maintainer, a FOSS org) helps land real first contributions at scale?

15. TESTING

- Domain and simulator logic stay pure and unit-tested; the simulator gets deterministic, seed-driven tests.
- CI: `vocab-lint`, `fmt`, `clippy -D warnings`, `cargo test`, plus a build-profile test asserting the `inside` profile compiles with no hardware/network code present.